
(deep into) Refactoring

Los refactoring de código son:

1. Cualquier transformación de código fuente
2. Cambios en el código que no alteran el comportamiento
3. Cambios en los datos del sistema para que funcione bien
4. Cambios en el código fuente que elimina code smells
5. Cambios en el código que mejoran el sistema
6. Transtornaciones del programador que arreglan el sistema
7. Cambios en el código que mantienen la funcionalidad
8. Mejoras en la funcionalidad sin cambios de código

Correctas: 2,7

Refactoring

- Transformaciones de código que preservan el comportamiento
- Preservar el comportamiento
 - (Hace lo que hacía antes) & (No hace lo que no hacía antes) *
- Se puede contar todas las maneras de escribir?
 - operación (función),
 - Clase
 - Librería, Framework
 - Programa
 - Elija su opción: Finito, Infinito contable o Infinito Incontable
- **Qué debería usarse para poder analizar (cualquier) código (dado un lenguaje de programación)?**

*Difícil... si requiere precisión terminamos en el universo de los métodos formales :-)

Refactorings

- Rename Variable
- Rename Method
- Extract Method
- Move Method to Component
- Replace Conditional with Polymorphism
- Replace temp with query



Qué debería usarse para poder analizar (cualquier) código ?

La herramienta secreta del
informático...

ABSTRACCIÓN

Qué debería usarse para poder analizar (cualquier) código ?

- **Abstracción**

- Creada desde el código
- Uniquivoca
- Manipulable por Algoritmos
- => Árboles !

- Syntax Trees

- Parsing
- Gramatica
- Abstract (AST)
- Concrete (CST)

EXPRESSION ::= NUMERAL | “(“NUMERAL”)

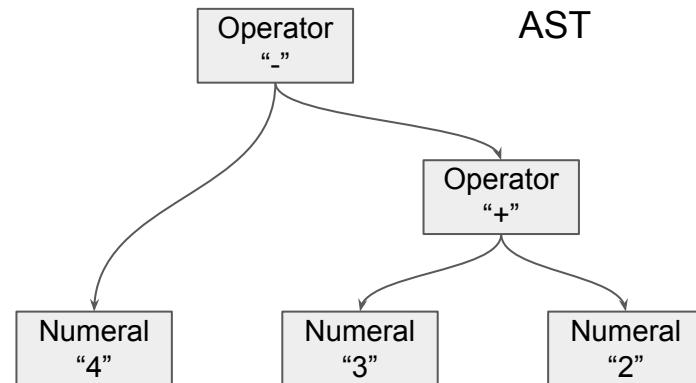
“EXPRESSION OPERATOR EXPRESSION”

OPERATOR ::= "+" | "-"

NUMERAL ::= DIGIT | DIGIT NUMERAL

DIGIT ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Ejemplo: $(4 - (3 + 2))$

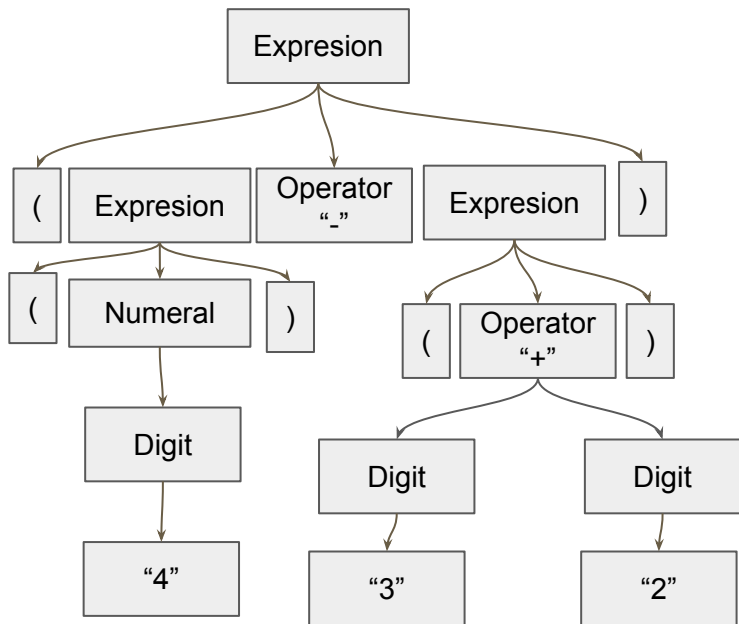


CST vs AST

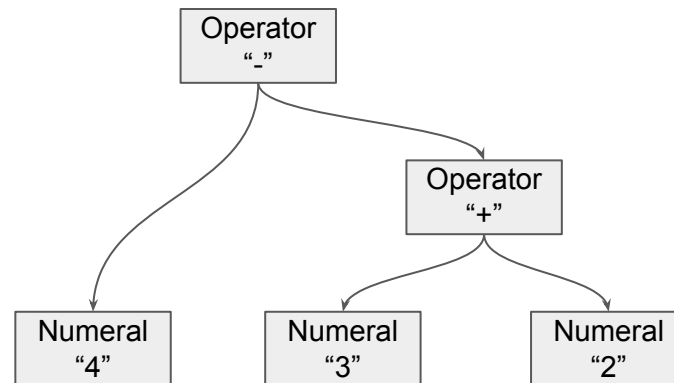
$((4) - (3 + 2))$

EXPRESSION ::= NUMERAL | "(" NUMERAL ")"
 | "(" EXPRESSION OPERATOR EXPRESSION ")"
OPERATOR ::= "+" | "-"
NUMERAL ::= DIGIT | DIGIT NUMERAL
DIGIT ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

Concrete ST: cada derivation rule es un subarbol



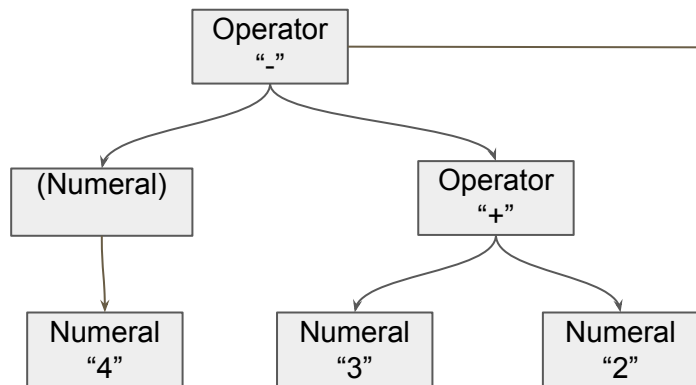
Abstract ST: estructura relevante



Refactoring(Tree)

- Transformación(Tree) = Tree'
 - Tree $\equiv$ Tree'

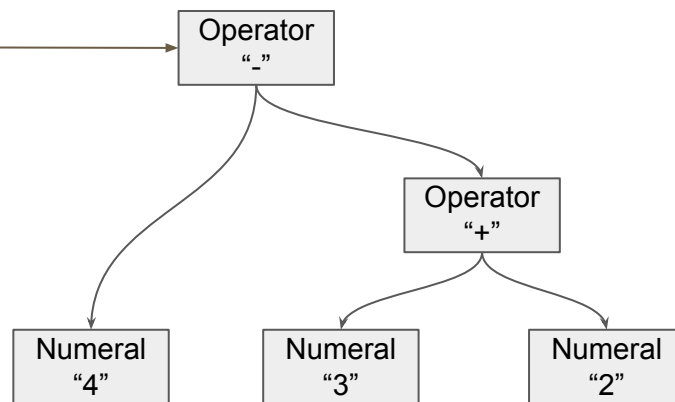
EXPRESSION ::= NUMERAL | (NUMERAL
 "(" EXPRESSION OPERATOR EXPRESSION ")"
(NUMERAL) ::= "(" NUMERAL ")"
OPERATOR ::= "+" | "-"
NUMERAL ::= DIGIT | DIGIT NUMERAL
DIGIT ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"



Pretty print

((4) - (3 + 2))

Clean Parenthesis



Pretty print

(4 - (3 + 2))

Refactoring SQL: alias de tablas

```
SELECT Orders.OrderID, Orders.OrderDate, Customers.CustomerName  
FROM Customers, Orders  
WHERE Customers.CustomerName='Around the Horn' AND Customers.CustomerID=Orders.CustomerID;
```

Alias: Customers → "c". Orders → "o"

1. Verificar precondiciones (ejemplo: $\nexists$ colisión de alias)
2. Agregar alias a la declaración de la tabla
3. Cambiar la ocurrencia de la tabla por su alias

```
SELECT o.OrderID, o.OrderDate, c.CustomerName  
FROM Customers AS c, Orders AS o  
WHERE c.CustomerName='Around the Horn' AND c.CustomerID=o.CustomerID;
```

Refactoring: alias “o” para Order

```
Select Orders.OrderID, Customers.CustomerName  
From Orders, Customers  
WHERE Customers.CustomerID = Orders.OrderID
```

```
Select Orders.OrderID, Customers.CustomerName  
From Orders as o, Customers  
WHERE Customers.CustomerID = Orders.OrderID
```

Correcto? Compila? Refactorizado?

```
Select Orders as o.OrderID, Customers.CustomerName  
From Orders as o, Customers  
WHERE Customers.CustomerID = Orders as o.OrderID
```

Correcto? Compila? Refactorizado?

Cómo sabemos que o no compila o no es refactoring

Estructura de la expresión

```
Select Orders.OrderID, Customers.CustomerName  
From Orders, Customers  
WHERE Customers.CustomerID = Orders.OrderID
```

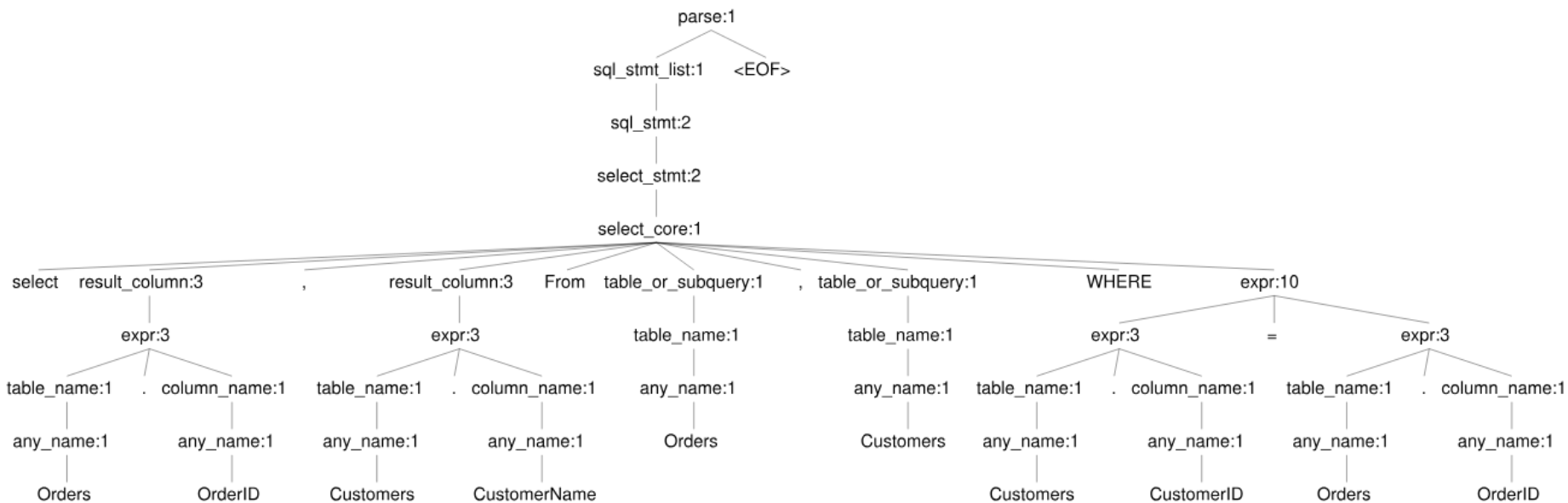
Orders (en azul) define el nombre de la tabla a utilizar

Orders (en verde) es una referencia a la tabla (que debe estar en la clausula From)

```
Select o.OrderID, Customers.CustomerName  
From Orders as o, Customers  
WHERE Customers.CustomerID = o.OrderID
```

Correcto: Compila, Refactorizado

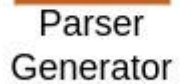
Select Orders.OrderID, Customers.CustomerName
From Orders, Customers
WHERE Customers.CustomerID = Orders.OrderID



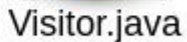


ANTLR (ANother Tool for Language Recognition)

- Generador de analizadores para leer, procesar, ejecutar o traducir texto estructurado o archivos binarios.
- Se usa ampliamente para crear lenguajes, herramientas y Frameworks.
- A partir de una gramática, ANTLR genera un analizador que puede construir y recorrer árboles de análisis.
- <http://lab.antlr.org/>



antlr



Proyecto Roo2

Antlr lab: herramienta de pruebas

Lexer Parser Sample

```
1  parser grammar ExprParser;
2  options { tokenVocab=ExprLexer; }
3
4  program
5      : stat EOF
6      | def EOF
7      ;
8
9  stat: ID '=' expr ';'
10     | expr ';'
11     ;
12
13 def : ID '(' ID (',' ID)* ')' '{' stat* '}' ;
14
15 expr: ID
16     | INT
17     | func
18     | 'not' expr
19     | expr 'and' expr
20     | expr 'or' expr
21     | expr SUM expr
22     | expr SUB expr
23     | expr MUL expr
24     | expr DIV expr
25     | expr POW expr
26     ;
27
28 func : ID '(' expr (',' expr)* ')' ;
```

Input sample.expr

```
1  f(x,y) {
2      a = 3+4+2;
3      x and y;
4  }
```

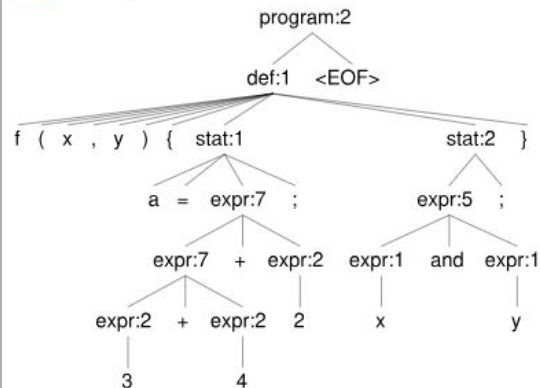
Start rule

program

Run

Show profiler

Tree Hierarchy



```

parser grammar ExprParser;
options { tokenVocab=ExprLexer; }

```

```

program
: stat EOF
| def EOF
;

```

```

stat: ID '=' expr ';'
| expr ';'
;

```

```

def : ID '(' ID (',' ID)* ')' '{ stat* }' ;

```

```

expr: ID
| INT
| func
| 'not' expr
| expr 'and' expr
| expr 'or' expr
| expr SUM expr
| expr SUB expr
| expr MUL expr
| expr DIV expr
| expr POW expr
;

```

```

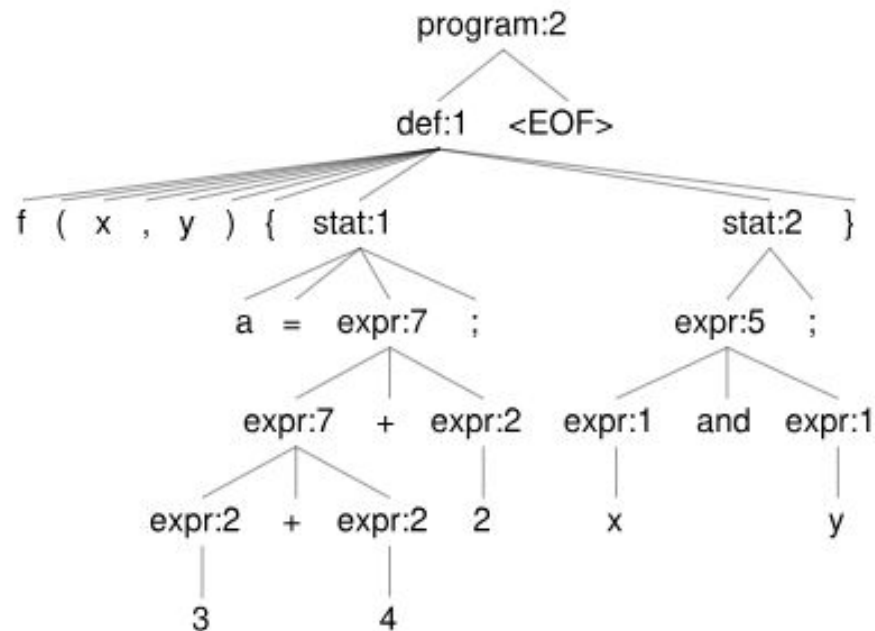
func : ID '(' expr (',' expr)* ')' ;

```

```

f(x,y) {
    a = 3+4+2;
    x and y;
}

```



```

parser grammar ExprParser;
options { tokenVocab=ExprLexer; }

```

```

program
: stat EOF
| def EOF
;

```

```

stat: ID '=' expr ';'
| expr ';'
;

```

```

def : ID '(' ID '(' ID* ')' '{ stat* }' ;

```

```

expr: ID
| INT
| func
| 'not' expr
| expr 'and' expr
| expr 'or' expr
| expr SUM expr
| expr SUB expr
| expr MUL expr
| expr DIV expr
| expr POW expr
;

```

```

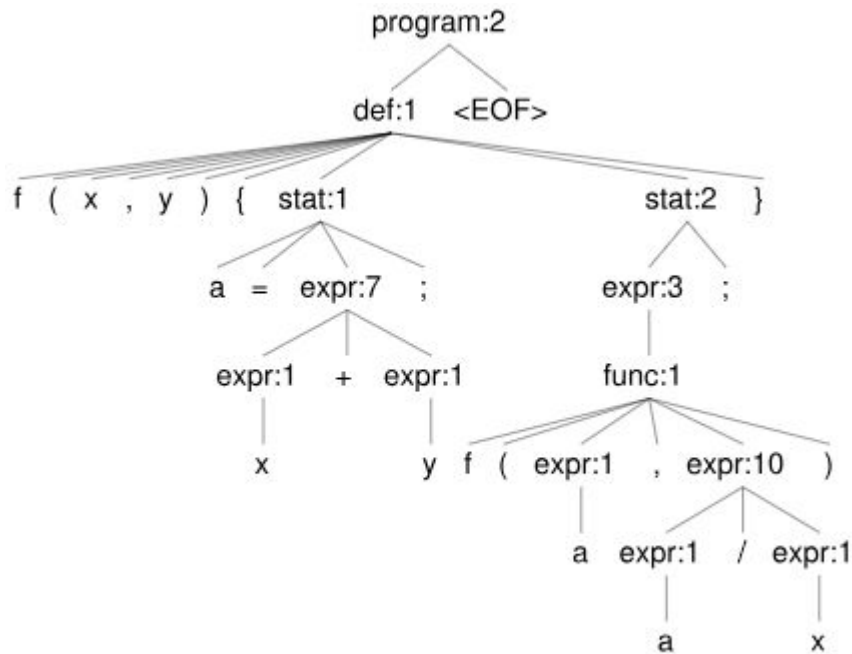
func : ID '(' expr '(' expr* ')' ;

```

```

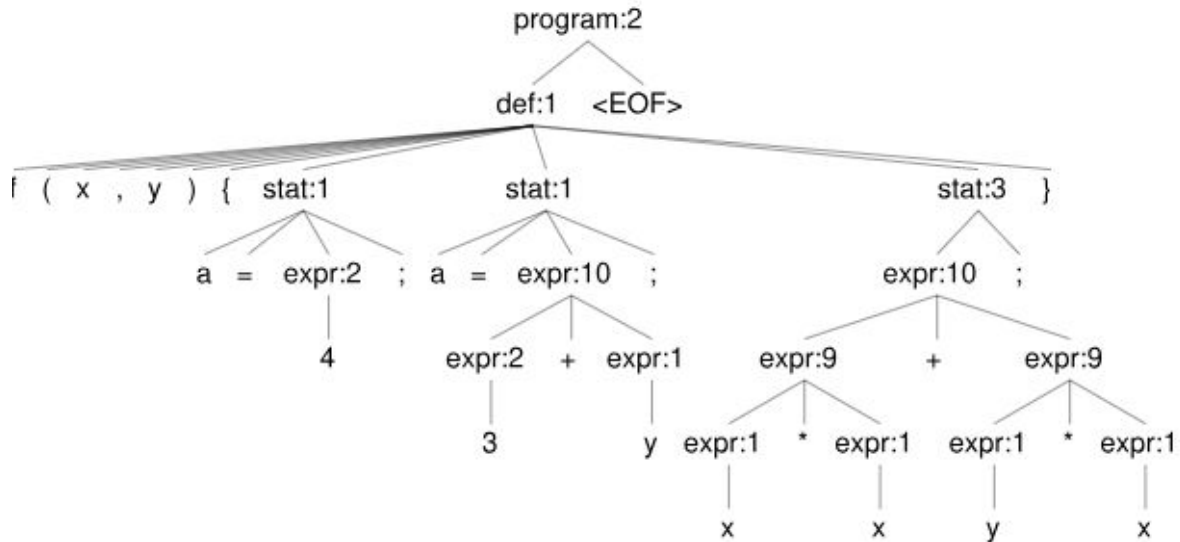
1  f(x,y) {
2      a = x + y;
3      f(a, a/x);
4  }
5

```



Variables no utilizadas

```
f(x,y) {  
  a = 4  
  a = 3 + y;  
  x * x + y * x;  
}
```



Algorithm LEVELORDER(T)

Input: root node T of a tree

Output: visits each node level by level

1. procedure LEVELORDER(T)
2. if T = NIL then
3. return
4. Q ← empty queue
5. ENQUEUE(Q, T)
6. while Q is not empty do
7. v ← DEQUEUE(Q)
8. **visit**(v)
9. for each child C of v, in order, do
10. ENQUEUE(Q, C)
11. end procedure

Visit(). Es un hook!

roo2.next()

- Code Smells del lenguaje Sample (disponible en antlr lab)
 - Version extendida por Roo2 (ver moodle)
- Refactoring
 - OO2 2026
 - Sample en base a los CST

